

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN,
RISET, DAN TEKNOLOGI
UNIVERSITAS BENGKULU – FAKULTAS TEKNIK
JURUSAN TEKNIK ELEKTRO

BUKU PETUNJUK PRAKTIKUM ANALISIS DERET WAKTU DAN PERAMALAN

Time Series Analysis and Forecasting in Electrical Engineering

Dosen Pengampu:
Ir. Novalio Daratha, S.T., M.Sc., Ph.D.

NIP. 197811102003121002

Laboratorium Sistem Tenaga Listrik & Komputasi Cerdas

BENGKULU
2026

Daftar Isi

Petunjuk Umum & Lingkungan Komputasi	1
1 Dekomposisi Sinyal Deret Waktu Kelistrikan	2
1.1 Dasar Teori	2
1.2 Langkah Praktikum & Kode Implementasi	2
1.3 Tugas & Laporan Praktikum 1	3
2 Uji Stasioneritas, Diferensiasi, dan Correlogram	4
2.1 Dasar Teori	4
2.2 Kode Implementasi Python	4
3 Metode Pemulusan Data (Holt-Winters)	6
3.1 Dasar Teori	6
3.2 Kode Implementasi Python	6
4 Pemodelan ARIMA dan SARIMA Box-Jenkins	7
4.1 Dasar Teori	7
4.2 Kode Implementasi Python	7
5 Model Eksogen Cuaca: SARIMAX dan CCF	8
5.1 Dasar Teori	8
5.2 Kode Implementasi Python	8
6 Discrete Kalman Filter untuk Sinyal Sensor Bising	9
6.1 Dasar Teori	9
6.2 Kode Implementasi Python (From Scratch)	9
7 Analisis Spektral FFT dan Dekomposisi Wavelet	11
7.1 Dasar Teori	11
7.2 Kode Implementasi Python	11
8 Feature Engineering dan Machine Learning (XGBoost)	12
8.1 Dasar Teori	12
8.2 Kode Implementasi Python	12
9 Deep Learning: Arsitektur LSTM dan Seq2Seq	13
9.1 Dasar Teori	13
9.2 Kode Implementasi PyTorch	13
10 Deteksi Anomali Konsumsi Energi Berbasis Autoencoder	14
10.1 Dasar Teori	14
10.2 Kode Implementasi Python	14

11 Integrasi Peramalan pada Operasi Sistem Tenaga	15
11.1 Dasar Teori	15
11.2 Tugas Simulasi Mandiri	15

Petunjuk Umum & Lingkungan Komputasi

Tujuan Praktikum

Praktikum Komputasi Analisis Deret Waktu bertujuan untuk membekali mahasiswa S1 Teknik Elektro dengan keterampilan teknis praktis dalam:

1. Mengolah dan memanipulasi dataset temporal skala besar sistem tenaga listrik, sensor IoT, serta pembangkit EBT.
2. Membangun *pipeline* pemodelan deret waktu dari metode klasik hingga *Deep Learning*.
3. Menginterpretasikan performa algoritma secara kuantitatif sesuai kaidah saintifik rekayasa elektro.

Kebutuhan Perangkat Lunak

Setiap mahasiswa diharapkan menggunakan lingkungan Python 3.10+ (atau GNU Octave / MATLAB) dengan pustaka standar berikut:

```
1 # Instalasi pustaka utama via terminal/command prompt
2 pip install numpy pandas scipy matplotlib statsmodels scikit-learn xgboost torch
   filterpy pywavelets
```

Bab 1

Dekomposisi Sinyal Deret Waktu Kelistrikan

Informasi Modul 1

Sub-CPMK 1: Mampu mengidentifikasi komponen tren, musiman, siklis, dan residu pada data kelistrikan serta memvisualisasikannya secara komputasional.

Alokasi Waktu: 170 Menit (Praktikum Mandiri & Asistensi Lab)

Dataset: Data beban listrik historis per-jam (`load_grid_hourly.csv`)

1.1 Dasar Teori

Deret waktu keteknikan y_t dapat didekomposisi menjadi tiga komponen deterministik dan satu komponen stokastik:

$$\text{Model Aditif: } y_t = T_t + S_t + R_t \quad \text{Model Multiplikatif: } y_t = T_t \times S_t \times R_t \quad (1.1)$$

di mana T_t adalah tren, S_t adalah musiman dengan periode m (misal $m = 24$ untuk data jam), dan R_t adalah residual (*irregular/noise*).

1.2 Langkah Praktikum & Kode Implementasi

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from statsmodels.tsa.seasonal import seasonal_decompose
4
5 # 1. Membaca dataset beban gardu induk
6 df = pd.read_csv('load_grid_hourly.csv', parse_dates=['timestamp'], index_col='
    timestamp')
7
8 # 2. Dekomposisi Klasik (Periode harian: 24 jam)
9 decomp = seasonal_decompose(df['load_mw'], model='additive', period=24)
10
11 # 3. Plot 4-Panel (Observed, Trend, Seasonal, Residual)
12 fig, axes = plt.subplots(4, 1, figsize=(12, 8), sharex=True)
13 decomp.observed.plot(ax=axes[0], color='navy', title='Observed Load (MW)')
14 decomp.trend.plot(ax=axes[1], color='darkred', title='Trend Component')
15 decomp.seasonal.plot(ax=axes[2], color='darkgreen', title='Daily Seasonality')
16 decomp.resid.plot(ax=axes[3], color='black', title='Residual / Noise')
17 plt.tight_layout()
18 plt.show()
```

1.3 Tugas & Laporan Praktikum 1

1. Hitunglah nilai variansi residual $\text{Var}(R_t)$ dan periksa apakah residual memiliki nilai rata-rata mendekati nol!
2. Lakukan dekomposisi multiplikatif pada data yang sama. Bandingkan nilai residual antara kedua model dan tentukan model mana yang paling cocok!

Bab 2

Uji Stasioneritas, Diferensiasi, dan Correlogram

Informasi Modul 2

Sub-CPMK 2: Mampu menghitung ACF/PACF, menguji stasioneritas (ADF, KPSS), dan melakukan transformasi diferensiasi data.

Dataset: Profil daya aktif transmisi (`active_power.csv`)

2.1 Dasar Teori

Sebagian besar model statistik linier mengasumsikan sinyal berada dalam keadaan stasioner kovarians:

$$\mathbb{E}[y_t] = \mu, \quad \text{Var}(y_t) = \sigma^2, \quad \text{Cov}(y_t, y_{t-k}) = \gamma_k \quad (2.1)$$

Uji *Augmented Dickey-Fuller* (ADF) menguji hipotesis nol (H_0) bahwa data memiliki *unit root* (non-stasioner). Jika p -value < 0.05 , H_0 ditolak (data stasioner).

2.2 Kode Implementasi Python

```
1 from statsmodels.tsa.stattools import adfuller, kpss
2 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
3
4 def check_stationarity(series):
5     result = adfuller(series.dropna())
6     print(f'ADF Statistic : {result[0]:.4f}')
7     print(f'p-value       : {result[1]:.4e}')
8     print('Status       : ', 'Stasioner' if result[1] < 0.05 else 'Non-Stasioner')
9
10 # Uji data asli vs diferensiasi ordo-1
11 print("--- Data Asli ---")
12 check_stationarity(df['power'])
13
14 df['diff_1'] = df['power'].diff()
15 print("\n--- Setelah Differencing (d=1) ---")
16 check_stationarity(df['diff_1'])
17
18 # Plot ACF dan PACF
19 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
20 plot_acf(df['diff_1'].dropna(), lags=40, ax=axes[0], title='Autocorrelation (ACF)',
21         )
22 plot_pacf(df['diff_1'].dropna(), lags=40, ax=axes[1], title='Partial ACF (PACF)')
```

22 | `plt.show()`

Bab 3

Metode Pemulusan Data (Holt-Winters)

Informasi Modul 3

Sub-CPMK 3: Mampu menerapkan metode pemulusan (*Moving Average*, Holt-Winters aditif/multiplikatif) untuk peramalan beban jangka pendek.

Dataset: Profil konsumsi listrik mingguan (`weekly_load.csv`)

3.1 Dasar Teori

Metode Holt-Winters mengakomodasi level (ℓ_t), tren (b_t), dan musiman (s_t):

$$\ell_t = \alpha(y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (3.1)$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1} \quad (3.2)$$

$$s_t = \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1 - \gamma)s_{t-m} \quad (3.3)$$

$$\hat{y}_{t+h|t} = \ell_t + hb_t + s_{t+h-m} \quad (3.4)$$

3.2 Kode Implementasi Python

```
1 from statsmodels.tsa.holtwinters import ExponentialSmoothing
2 from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error
3 import numpy as np
4
5 # Split Train dan Test (24 jam terakhir sebagai test set)
6 train, test = df['load'][:-24], df['load'][-24:]
7
8 # Fitting Model Holt-Winters Aditif
9 model = ExponentialSmoothing(train, seasonal_periods=24, trend='add', seasonal='
    add').fit()
10 forecast = model.forecast(24)
11
12 # Evaluasi Performa
13 rmse = np.sqrt(mean_squared_error(test, forecast))
14 mape = mean_absolute_percentage_error(test, forecast) * 100
15 print(f'RMSE: {rmse:.2f} MW | MAPE: {mape:.2f}%')
```

Bab 4

Pemodelan ARIMA dan SARIMA Box-Jenkins

Informasi Modul 4 & 5

Sub-CPMK 4 & 5: Mampu menyusun ordo p, d, q, P, D, Q dan mengestimasi model SARIMA untuk data berkala musiman serta memvalidasi residual.

Dataset: Data time series gardu distribusi (`substation_load.csv`)

4.1 Dasar Teori

Model SARIMA disimbolkan dengan $(p, d, q) \times (P, D, Q)_s$:

$$\Phi_P(B^s)\phi_p(B)(1-B)^d(1-B^s)^D y_t = \Theta_Q(B^s)\theta_q(B)\epsilon_t \quad (4.1)$$

4.2 Kode Implementasi Python

```
1 from statsmodels.tsa.statespace.sarimax import SARIMAX
2 from statsmodels.stats.diagnostic import acorr_ljungbox
3
4 # 1. Definisi dan Pelatihan Model SARIMA(1,1,1)(1,1,1)_24
5 sarima_model = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 24),
6                         enforce_stationarity=False, enforce_invertibility=False)
7 results = sarima_model.fit(dispatch=False)
8
9 print(results.summary())
10
11 # 2. Diagnostik Residual (Ljung-Box Test untuk White Noise)
12 lb_test = acorr_ljungbox(results.resid, lags=[10, 20], return_df=True)
13 print("\n--- Ljung-Box Test Results ---")
14 print(lb_test)
15
16 # 3. Prediksi & Plot
17 pred = results.get_forecast(steps=24)
18 pred_ci = pred.conf_int()
```

Bab 5

Model Eksogen Cuaca: SARIMAX dan CCF

Informasi Modul 6

Sub-CPMK 6: Mampu membangun model SARIMAX dengan mengintegrasikan variabel suhu udara dan radiasi matahari.

Dataset: Beban gedung & data stasiun cuaca (`hvac_weather.csv`)

5.1 Dasar Teori

Variabel eksogen X_t (seperti temperatur) mempengaruhi konsumsi daya AC secara langsung:

$$y_t = \beta X_t + \frac{\Theta(B)}{\Phi(B)} \epsilon_t \quad (5.1)$$

5.2 Kode Implementasi Python

```
1 # Integrasi variabel temperatur (X_train & X_test)
2 sarimax_model = SARIMAX(train['load'], exog=train[['temperature', 'humidity']],
3                           order=(1, 1, 1), seasonal_order=(1, 1, 0, 24))
4 res_exog = sarimax_model.fit(dispatch=False)
5
6 # Prediksi menggunakan fitur eksogen masa depan
7 forecast_exog = res_exog.forecast(steps=24, exog=test[['temperature', 'humidity']])
```

Bab 6

Discrete Kalman Filter untuk Sinyal Sensor Bising

Informasi Modul 7

Sub-CPMK 7: Mampu memformulasikan model State-Space dan menerapkan Discrete Kalman Filter untuk memulihkan sinyal sensor bising.

Dataset: Sinyal telemetry arus DC dan tegangan (`noisy_sensor.csv`)

6.1 Dasar Teori

Persamaan Ruang Keadaan Diskrit:

$$x_k = Ax_{k-1} + Bu_{k-1} + w_{k-1}, \quad w \sim \mathcal{N}(0, Q) \quad (6.1)$$

$$z_k = Hx_k + v_k, \quad v \sim \mathcal{N}(0, R) \quad (6.2)$$

Algoritma Kalman Filter bekerja dalam 2 fase: **Time Update (Prediksi)** dan **Measurement Update (Koreksi)**.

6.2 Kode Implementasi Python (From Scratch)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameter Sistem 1D
5 A = 1.0; H = 1.0; Q = 1e-4; R = 0.05**2
6 x_hat = 0.0; P = 1.0
7
8 measurements = df['voltage_noisy'].values
9 estimates = []
10
11 for z in measurements:
12     # 1. Time Update (Prediksi)
13     x_hat_minus = A * x_hat
14     P_minus = A * P * A + Q
15
16     # 2. Measurement Update (Koreksi)
17     K = P_minus * H / (H * P_minus * H + R)
18     x_hat = x_hat_minus + K * (z - H * x_hat_minus)
19     P = (1 - K * H) * P_minus
20
21     estimates.append(x_hat)
```

```
22 |  
23 | df['filtered_voltage'] = estimates
```

Bab 7

Analisis Spektral FFT dan Dekomposisi Wavelet

Informasi Modul 8

Sub-CPMK 8: Mampu menganalisis spektrum frekuensi sinyal deret waktu menggunakan FFT dan dekomposisi Wavelet untuk deteksi transien/harmonisa.

Dataset: Sinyal getaran motor induksi & arus transien (`motor_vibration.csv`)

7.1 Dasar Teori

Transformasi Wavelet Diskrit (*Discrete Wavelet Transform* - DWT) menguraikan sinyal ke dalam koefisien aproksimasi (cA) dan detail (cD):

$$y(t) = \sum_k cA_{J,k} \phi_{J,k}(t) + \sum_{j=1}^J \sum_k cD_{j,k} \psi_{j,k}(t) \quad (7.1)$$

7.2 Kode Implementasi Python

```
1 import pywt
2 import numpy as np
3
4 signal = df['current_transient'].values
5
6 # Dekomposisi Wavelet 3 Level menggunakan wavelet Daubechies (db4)
7 coeffs = pywt.wavedec(signal, 'db4', level=3)
8 cA3, cD3, cD2, cD1 = coeffs
9
10 # Denoising dengan Soft Thresholding pada koefisien detail
11 threshold = np.median(np.abs(cD1)) / 0.6745 * np.sqrt(2 * np.log(len(signal)))
12 coeffs_thresh = [cA3] + [pywt.threshold(c, threshold, mode='soft') for c in [cD3,
13                                     cD2, cD1]]
14
15 # Rekonstruksi Sinyal Bersih
16 clean_signal = pywt.waverec(coeffs_thresh, 'db4')
```

Bab 8

Feature Engineering dan Machine Learning (XGBoost)

Informasi Modul 9

Sub-CPMK 9: Mampu merancang rekayasa fitur temporal dan menerapkan algoritma XGBoost/LightGBM untuk peramalan beban listrik jangka pendek.

Dataset: Dataset beban listrik multivariat (`power_features.csv`)

8.1 Dasar Teori

Mengubah masalah peramalan sekuensial menjadi regresi terawasi (*Supervised Learning*) melalui pembuatan fitur:

$$y_t = f(y_{t-1}, y_{t-2}, y_{t-24}, \bar{y}_{\text{roll}_{24}}, \sin(2\pi \cdot \text{hour}/24), \cos(2\pi \cdot \text{hour}/24), \dots) \quad (8.1)$$

8.2 Kode Implementasi Python

```
1 import xgboost as xgb
2
3 # 1. Feature Engineering
4 def create_features(df):
5     df['hour'] = df.index.hour
6     df['dayofweek'] = df.index.dayofweek
7     df['lag_1'] = df['load'].shift(1)
8     df['lag_24'] = df['load'].shift(24)
9     df['lag_168'] = df['load'].shift(168)
10    df['rolling_mean_24'] = df['load'].shift(1).rolling(window=24).mean()
11    return df.dropna()
12
13 df_feat = create_features(df)
14 X = df_feat[['hour', 'dayofweek', 'lag_1', 'lag_24', 'lag_168', 'rolling_mean_24']]
15 y = df_feat['load']
16
17 # 2. Training XGBoost Regressor
18 X_train, X_test = X[:-168], X[-168:]
19 y_train, y_test = y[:-168], y[-168:]
20
21 model_xgb = xgb.XGBRegressor(n_estimators=500, learning_rate=0.03, max_depth=6)
22 model_xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], early_stopping_rounds=30, verbose=False)
```

Bab 9

Deep Learning: Arsitektur LSTM dan Seq2Seq

Informasi Modul 10 & 11

Sub-CPMK 10 & 11: Mampu merancang tensor 3D sekuensial dan melatih arsitektur LSTM / Seq2Seq untuk peramalan daya pembangkit surya (PV).

Dataset: Data radiasi & daya PLTS (`solar_pv_generation.csv`)

9.1 Dasar Teori

Jaringan *Long Short-Term Memory* (LSTM) mengatur aliran memori dengan tiga gerbang utama: *Forget Gate* (f_t), *Input Gate* (i_t), dan *Output Gate* (o_t).

9.2 Kode Implementasi PyTorch

```
1 import torch
2 import torch.nn as nn
3
4 class LSTMForecaster(nn.Module):
5     def __init__(self, input_dim=4, hidden_dim=64, num_layers=2, output_dim=24):
6         super(LSTMForecaster, self).__init__()
7         self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True,
8                               dropout=0.2)
9         self.fc = nn.Linear(hidden_dim, output_dim)
10
11     def forward(self, x):
12         # x shape: (batch_size, seq_length, input_dim)
13         out, (hn, cn) = self.lstm(x)
14         # Ambil hidden state terakhir
15         out = self.fc(out[:, -1, :])
16         return out
17
18 # Inisialisasi Model, Loss, dan Optimizer
19 model = LSTMForecaster()
20 criterion = nn.MSELoss()
21 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```


Bab 10

Deteksi Anomali Konsumsi Energi Berbasis Autoencoder

Informasi Modul 12

Sub-CPMK 12: Mampu membangun model rekonstruksi LSTM Autoencoder untuk deteksi pencurian listrik dan anomali sensor meter pintar.

Dataset: Profil konsumsi Smart Meter (`smartmeter_anomaly.csv`)

10.1 Dasar Teori

Autoencoder dilatih hanya pada data operasi normal. Ketika anomali terjadi, galat rekonstruksi (*Reconstruction Error* $\mathcal{L}$) akan melonjak melampaui ambang batas τ :

$$\mathcal{L}(x_t, \hat{x}_t) = \|x_t - \hat{x}_t\|^2 > \tau \quad (10.1)$$

10.2 Kode Implementasi Python

```
1 # Menghitung Reconstruction Error (MSE per sampel deret waktu)
2 reconstructed = autoencoder.predict(X_test)
3 mse_errors = np.mean(np.power(X_test - reconstructed, 2), axis=(1, 2))
4
5 # Menentukan ambang batas anomali (Threshold = Mean + 3*Std)
6 threshold = np.mean(mse_train_errors) + 3 * np.std(mse_train_errors)
7 anomalies = mse_errors > threshold
8
9 print(f'Jumlah titik anomali terdeteksi: {np.sum(anomalies)} dari {len(mse_errors)}
    } jendela waktu')
```

Bab 11

Integrasi Peramalan pada Operasi Sistem Tenaga

Informasi Modul 13 & 14

Sub-CPMK 13: Mampu mengintegrasikan peramalan beban & EBT ke dalam simulasi penjadwalan baterai (BESS) pada microgrid berbasis ketidakpastian.

Dataset: Profil gabungan PV, Beban, dan Tarif Listrik (`microgrid.dispatch.csv`)

11.1 Dasar Teori

Keseimbangan daya pada mikrogrid dengan sistem penyimpanan baterai (BESS):

$$P_{\text{grid}}(t) = \hat{P}_{\text{load}}(t) - \hat{P}_{\text{PV}}(t) + P_{\text{charge}}(t) - P_{\text{discharge}}(t) \quad (11.1)$$

Tujuan optimasi adalah meminimalkan biaya pembelian energi listrik dari jaringan utama dengan memanfaatkan perkiraan daya surya saat jam puncak.

11.2 Tugas Simulasi Mandiri

1. Bangun model peramalan probabilistik (Quantile Regression 10%, 50%, dan 90%) untuk membatasi ketidakpastian daya fotovoltaiik.
2. Simulasikan strategi operasi BESS (*Peak Shaving*) selama 24 jam dan hitung penghematan biaya listrik yang diperoleh!